

Heverton Eduardo Peres

LSP Integrado: Inteligência de Código em Cada Escrita & Debug com DAP: O Agente como Debugger

Obra técnica de literatura especializada, produzida e diagramada conforme as normas ABNT para publicação editorial.

Brasil
2026

Sumário

LSP Integrado: Inteligência de Código em Cada Escrita	3
Inteligência embarcada no estaleiro	3
O que é o LSP e por que ele importa	3
As 14 operações LSP	4
Operações de documento	4
Operações de workspace	4
A magia do rename com willRenameFiles	5
Auto-deteção e configuração	5
Diagnostics: inspeção em tempo real	6
Hover: especificações de um símbolo	6
Definition e References: mapeando dependências	6
Rename: a operação mais poderosa	7
CodeAction: correções inteligentes	7
Completion e SignatureHelp: assistência contextual	7
Configurando LSP servers	8
O rename que quebrou tudo vs. o rename seguro	8
Armadilhas comuns	9
Próximos Passos	9
Debug com DAP: O Agente como Debugger	11
Diagnóstico profundo do motor	11
O que é o DAP e por que ele existe	11
Os três debuggers do OMP	12
O fluxo de uma sessão de debug	12
O debugger como um câmbio lento	12
Inicializando a sessão DAP	13
Conectando a um processo Python com debugpy	13
Debugando C com lldb-dap	14
Debugando Go com dlvsrv	15
As 28 operações DAP em detalhe	15
A falha silenciosa em produção	16
Armadilhas comuns ao usar DAP	17
Próximos Passos	17

LSP Integrado: Inteligência de Código em Cada Escrita

Inteligência embarcada no estaleiro

No capítulo anterior, você dominou as edições *hashline* — o sistema que permite ao agente apontar para blocos de código usando hashes em vez de retratar linhas inteiras, economizando até 61% de tokens de saída.

Mas edits precisos não são suficientes quando o agente precisa ENTENDER o código antes de mexer nele. É aqui que entra o LSP — o Language Server Protocol — a mesma tecnologia que faz o VS Code autocompletar, mostrar erros em tempo real e renomear símbolos em todo o projeto.

O que é o LSP e por que ele importa

O Language Server Protocol (LSP) é um protocolo padronizado que separa a inteligência de código do editor. Antes do LSP, cada editor precisava implementar sua própria integração com cada linguagem — um trabalho hercúleo e duplicado. Com o LSP, um único servidor de linguagem serve qualquer editor que suporte o protocolo.

Pense no LSP como um conjunto de especialistas embarcados no estaleiro. Cada um conhece profundamente uma linguagem e pode responder perguntas como: “Onde essa função é definida?”, “Quais arquivos usam esse símbolo?”, “Esse código tem erros?”, “Como posso renomear isso de forma segura?”.

Antes do LSP, esses especialistas só existiam dentro do IDE. No OMP, eles estão disponíveis para o agente em cada interação.

As 14 operações LSP

O OMP expõe 14 operações LSP organizadas em duas categorias: operações de documento (que analisam um arquivo específico) e operações de workspace (que varrem o projeto inteiro).

Operações de documento

1. **diagnostics** — erros e warnings em tempo real, como um inspetor que sinaliza falhas estruturais no casco.
2. **hover** — informações sobre um símbolo ao passar o cursor, como uma placa de especificações de um equipamento.
3. **definition** — localiza onde um símbolo é definido, como um mapa mostrando a origem de cada peça.
4. **implementation** — encontra todas as implementações de uma interface, como um inventário de todas as variantes de um componente.
5. **typeDefinition** — mostra o tipo de um símbolo, como a ficha técnica detalhada de um material.
6. **completion** — sugere código contextualmente, como um catálogo de peças compatíveis.
7. **signatureHelp** — mostra os parâmetros de uma função durante a digitação, como um manual aberto na página certa.
8. **formatting** — formata o código conforme padrões, como um alinhador que deixa tudo no padrão do estaleiro.
9. **codeLens** — exibe informações contextuais inline (número de referências, testes), como indicadores painel no navio.
10. **documentSymbol** — lista todos os símbolos de um arquivo, como o índice de peças de um navio.

Operações de workspace

11. **references** — encontra todas as ocorrências de um símbolo no projeto, como um radar que detecta todas as dependências.
12. **workspaceSymbol** — busca símbolos em todo o workspace, como um sistema de GPS que localiza qualquer componente.
13. **rename** — renomeia um símbolo em todos os arquivos de forma segura, como um engenheiro que atualiza todos os registros antes de mudar o nome de uma peça.

14. **codeAction** — sugere correções e refactorings, como um consultor que recomenda melhorias com base no estado atual.

A magia do rename com willRenameFiles

O rename é provavelmente a operação LSP mais poderosa no dia a dia do agente. Quando o OMP pede um rename, o fluxo não é apenas “trocar o nome em todos os arquivos”.

O protocolo workspace/willRenameFiles garante que re-exports, barrel files e imports com alias sejam atualizados ANTES do arquivo ser movido.

Isso é fundamental: sem willRenameFiles, um rename poderia quebrar imports em módulos que o agente nem conhece. Com o protocolo, o servidor LSP percorre toda a cadeia de dependências e ajusta tudo antes que a mudança aconteça.

É como ter um engenheiro que verifica todos os navios afetados antes de remover uma peça do estaleiro.

Auto-deteção e configuração

O OMP não exige configuração manual para a maioria dos cenários. O sistema de auto-deteção verifica duas condições.

Primeiro, o diretório de trabalho contém pelo menos um dos `rootMarkers` do servidor (como `package.json` para TypeScript, `Cargo.toml` para Rust, `go.mod` para Go).

Segundo, o binário do servidor está disponível — primeiro em diretórios locais do projeto (`node_modules/.bin/`, ambientes virtuais Python), depois no `$PATH`.

Quando ambas as condições são atendidas, o servidor inicia automaticamente. Para projetos que precisam de ajustes, a hierarquia de configuração permite overrides em diferentes níveis.

Global: `~/.lsp.json` ou `~/.omp/agent/lsp.json`

Projeto: `<cwd>/lsp.json`

Raiz: `<cwd>/lsp.json`

Cada nível herda do anterior e sobrepõe apenas os campos especificados — configuração merge shallow por servidor.

Diagnostics: inspeção em tempo real

Quando o agente precisa saber se um arquivo tem erros, ele chama diagnostics. O servidor LSP analisa o arquivo e retorna uma lista de problemas — erros de sintaxe, warnings de tipos, inconsistências de formatação.

```
{
  "operation": "diagnostics",
  "file": "src/main.rs"
}
```

O resultado contém a severidade (error, warning, info), a posição exata e a mensagem. É como ter um inspetor de quality assurance trabalhando 24/7 no estaleiro.

Hover: especificações de um símbolo

Ao passar o cursor sobre um símbolo, o hover retorna o tipo, a documentação e a assinatura. No OMP, isso se traduz em uma chamada que o agente pode fazer antes de decidir como usar uma função.

```
{
  "operation": "hover",
  "file": "src/utils.rs",
  "line": 42,
  "character": 15
}
```

É como consultar a ficha técnica de uma peça antes de instalá-la — você sabe exatamente o que está mexendo.

Definition e References: mapeando dependências

Definition encontra onde um símbolo é definido. References encontra onde ele é usado. Juntos, eles dão ao agente o mapa completo de dependências.

No estaleiro, é como ter um sistema de rastreamento que mostra de onde veio cada peça e quais navios ela afeta — informação crucial antes de qualquer modificação.

Rename: a operação mais poderosa

O rename é onde a integração do OMP realmente brilha. Quando o agente pede um rename, o servidor LSP usa `workspace/willRenameFiles` para garantir que todos os arquivos afetados sejam atualizados ANTES da mudança.

```
{
  "operation": "rename",
  "file": "src/utils/format.ts",
  "line": 5,
  "character": 10,
  "newName": "formatBytes"
}
```

O resultado é uma lista de edits — cada um correspondendo a um arquivo que precisa ser alterado. Re-exports, barrel files, imports com alias, tudo atualizado de forma atômica.

CodeAction: correções inteligentes

CodeAction analisa um trecho de código e sugere correções. Pode ser desde “importar o símbolo que está faltando” até “extrair método” ou “adicionar tipo de retorno”.

```
{
  "operation": "codeAction",
  "file": "src/main.ts",
  "line": 15,
  "character": 5,
  "endLine": 20,
  "endCharacter": 1,
  "only": ["quickfix", "refactor"]
}
```

É como ter um consultor sênior que olha para o código e diz: “Essa função poderia ser simplificada”, “Esse import está faltando”, “Esse tipo deveria ser explícito”.

Completion e SignatureHelp: assistência contextual

Completion oferece sugestões de código baseadas no contexto. SignatureHelp mostra os parâmetros de uma função enquanto o agente digita.

No estaleiro, é como ter um catálogo de peças que se atualiza automaticamente mostrando quais componentes são compatíveis com o que já está instalado.

Configurando LSP servers

Para linguagens não suportadas pela auto-deteção, ou para projetos com necessidades específicas, a configuração é direta.

```
{
  "servers": {
    "my-custom-lsp": {
      "command": "my-lsp-server",
      "args": ["--stdio"],
      "fileTypes": [".xyz"],
      "rootMarkers": [".xyz-project", ".git"]
    }
  }
}
```

Para desabilitar um servidor built-in em um projeto específico, basta adicionar `disabled: true` na configuração do projeto.

Para ajustar configurações de um servidor existente, basta sobrescrever os campos desejados — a configuração faz merge shallow.

O rename que quebrou tudo vs. o rename seguro

Imagine que você pediu ao agente para renomear uma função `processData` para `transformPayload` em um projeto TypeScript com 15 arquivos.

Sem LSP, o agente faria um find-and-replace simples — trocando o nome em todos os arquivos. Mas e se um arquivo re-exporta a função com um alias? E se um barrel file indexa o módulo? E se um import usa `import { processData as pd }`? O find-and-replace simples ignora tudo isso e quebra o projeto silenciosamente.

Agora veja o que acontece com o LSP integrado do OMP. O agente chama a operação `rename`, o servidor LSP usa `workspace/willRenameFiles` para mapear TODAS as dependências — incluindo re-exports com alias, barrel files e imports dinâmicos — e gera uma lista de edits atômicos.

Cada arquivo é atualizado corretamente antes que o próximo seja processado. O resultado? Zero quebras, zero imports órfãos, zero erros de compilação.

Essa é a diferença entre ter um estagiário que faz find-and-replace e ter um engenheiro de confiabilidade que percorre toda a cadeia de dependências antes de mudar qualquer coisa.

Armadilhas comuns

Auto-deteção não encontrou o servidor. Verifique se o `rootMarker` do servidor existe na raiz do projeto. Se o projeto usa uma estrutura não padrão, crie um `.omp/lsp.json` com os rootMarkers corretos.

Conflito entre servidores para a mesma linguagem. O OMP aceita múltiplos servidores para a mesma linguagem. Para evitar conflitos, desabilite o que não usa via `disabled: true` na configuração do projeto.

Rename quebra imports dinâmicos. O rename LSP não consegue rastrear imports dinâmicos (`import()`) — eles dependem de strings em runtime. Nesses casos, o agente deve complementar com grep para localizar e ajustar manualmente.

Servidor não inicializa. Se o binário do servidor não está no PATH nem nos diretórios locais do projeto, a auto-deteção falha silenciosamente. Use `lsp` com a operação `diagnostics` em um arquivo para verificar se o servidor está ativo.

Próximos Passos

Neste capítulo, você conheceu as 14 operações LSP integradas ao OMP — desde `diagnostics` e `hover` até `rename` e `codeAction` — e entendeu como elas transformam a capacidade do agente de entender e modificar código com precisão.

Os três pontos que você deve levar deste capítulo: o LSP dá ao agente a mesma inteligência que o IDE — diagnósticos, navegação, renames seguros; a integração com `willRenameFiles` é o que separa um rename confiável de um quebrador de projeto; e a auto-deteção e a configuração hierárquica tornam o setup praticamente transparente para a maioria dos projetos.

No próximo capítulo, você vai dar um passo além da leitura e escrita de código: vamos explorar como o OMP dirige debuggers reais — `lldb` para C, `dlv` para Go, `debugpy` para Python — através da ferramenta DAP.

Acesse a documentação completa: <https://omp.sh/docs>

Siga-nos nas redes sociais para dicas, tutoriais e novidades sobre o Oh My Pi.

Debug com DAP: O Agente como Debugger

Diagnóstico profundo do motor

No capítulo anterior, você dominou o LSP integrado — 14 operações que dão ao agente a inteligência do seu IDE. Agora vamos um passo além: não apenas ler e renomear código, mas pausar sua execução no exato ponto onde o problema acontece, inspecionar variáveis em tempo real e corrigir o bug na raiz.

Imagine que, em vez de apenas olhar para o casco do navio por fora, você pudesse abrir uma escotilha, descer ao porão e examinar cada parafuso do motor enquanto ele ainda está ligado. É exatamente isso que o Debug Adapter Protocol (DAP) permite ao agente fazer.

O que é o DAP e por que ele existe

Antes do DAP, cada IDE precisava escrever sua própria integração com cada debugger. O VS Code tinha uma forma de falar com o GDB, o Vim tinha outra, e os debuggers precisavam suportar dezenas de protocolos diferentes.

Foi como se cada estaleiro tivesse seu próprio sistema de comunicação interna — a tripulação não conseguia falar entre si quando trocava de navio.

O DAP resolve isso criando um protocolo único. Assim como um standard de encaixe permite que qualquer equipamento de qualquer fabricante se conecte ao mesmo casco, o DAP permite que qualquer IDE fale com qualquer debugger usando a mesma linguagem.

O OMP implementa as 28 operações desse protocolo, o que significa que ele pode dirigir debuggers de verdade — não simuladores, não print statements, debuggers reais que pausam a execução e permitem inspeção completa do estado.

Os três debuggers do OMP

O OMP se conecta a três engines de debug diferentes, cada um especializado em uma linguagem.

lldb-dap: o debugger da família LLVM/Clang, otimizado para C, C++ e Objective-C. Ele é o equipamento pesado do estaleiro, capaz de examinar memória bruta, registros da CPU e estruturas de dados de baixo nível.

dlv (Delve): o debugger nativo da Go. Se o lldb é o guindaste de casco, o dlv é o scanner de motor — ele entende goroutines, canais e o runtime da Go, coisas que um debugger genérico simplesmente não enxerga.

debugpy: o debugger do Python, compatível com o protocolo DAP. Ele permite pausar scripts Python, inspecionar objetos dinâmicos e avaliar expressões em tempo de execução — como um instrumento de medição que se adapta automaticamente ao tipo de peça que está analisando.

O fluxo de uma sessão de debug

Toda sessão de debug segue um ciclo comum.

Primeiro, o agente inicializa a sessão DAP, informando ao debugger quais capabilities ele suporta. Depois, ele pode lançar um novo processo ou se anexar a um processo já em execução.

Uma vez conectado, o agente define pontos de parada, dá o comando de continuação e espera o debugger reportar que atingiu um desses pontos.

É nesse momento que a mágica acontece: o agente pode ler variáveis, caminhar pela pilha de chamadas e até mesmo avaliar expressões arbitrárias no contexto do programa pausado.

O debugger como um câmbio lento

Pense no debugging como inspecionar um navio em construção no estaleiro. Quando você constrói um navio, às vezes ele não funciona como esperado — o motor faz um barulho estranho, a rota desvia, ou uma peça simplesmente não encaixa.

Na vida real, você desligaria o motor, abriria a cobertura e examinaria cada componente com uma lanterna e um multímetro.

O DAP faz exatamente isso, mas no mundo digital. Em vez de desligar o motor inteiramente, o debugger coloca um “câmbio lento” — ele pausa a execução no ponto exato que você quer examinar, sem matar o processo.

É como congelar o tempo dentro do navio para que a tripulação possa caminhar pelo casco e verificar cada solda, cada parafuso, cada fio elétrico sem que o motor pare de funcionar quando o tempo voltar ao normal.

O OMP, nesse cenário, é o Mestre de Estaleiro que comanda essa inspeção. Ele decide onde colocar os pontos de verificação (breakpoints), quando avançar um passo (step), e quais instrumentos usar para medir (scopes e variables).

Iniciando a sessão DAP

A ferramenta `debug` do OMP é a porta de entrada para tudo. Quando você pede ao agente que debugge algo, ele abre uma sessão DAP e se conecta ao debugger apropriado.

```
# Exemplo: como o OMP se conecta ao debugpy para Python
debug_session = {
    "command": "initialize",
    "arguments": {
        "clientID": "omp",
        "adapterID": "debugpy",
        "supportsProgressReporting": True,
        "supportsRunInTerminalRequest": True
    }
}
```

O debugger responde com suas capabilities — quais tipos de breakpoints ele suporta, se aceita eval de expressões, se pode listar threads. Essa negociação é automática; o OMP cuida de tudo para você.

Conectando a um processo Python com debugpy

Vamos ver como o OMP debuga um script Python. Primeiro, você precisa que o debugpy esteja instalado no seu ambiente. Depois, o agente pode lançar o script ou se anexar a um processo que já está rodando.

```
# script_com_bug.py
def calcular_media(notas):
```

```

total = 0
for nota in notas:
    total += nota
media = total / len(notas) # Bug: divisao por zero se notas for vazio
return media

# Chamada que causa o erro
resultado = calcular_media([])
print(f"Media: {resultado}")

```

Quando o OMP detecta esse bug, ele pode lançar uma sessão de debug automaticamente. O agente diz ao debugpy para pausar na linha da divisão e, quando o programa atinge esse ponto, inspeciona a variável `notas` — descobrindo que é uma lista vazia, o que causa a divisão por zero.

```

# Comando debug que o OMP envia ao debugpy
set_breakpoints = {
    "command": "setBreakpoints",
    "arguments": {
        "source": {"path": "/caminho/para/script_com_bug.py"},
        "breakpoints": [
            {"line": 5, "condition": "len(notas) == 0"}
        ]
    }
}

# Quando o breakpoint e atingido, o agente inspeciona
evaluate_expr = {
    "command": "evaluate",
    "arguments": {
        "expression": "notas",
        "frameId": 1,
        "context": "watch"
    }
}

# Retorna: {"result": "[]", "type": "list"}

```

Debugando C com lldb-dap

Para código nativo, o OMP usa o lldb-dap. A diferença principal é que você precisa compilar o binário com símbolos de debug (`-g`) antes de iniciar a sessão.

```
# Compilar com símbolos de debug
gcc -g -o meu_programa meu_programa.c
```

Uma vez rodando, o agente pode definir breakpoints em funções específicas, examinar o frame da pilha e ler variáveis locais — tudo em código nativo, sem precisar de um IDE gráfico.

Debugando Go com dlv

O Delve (dlv) é o debugger nativo da Go e entende profundamente o runtime da linguagem. Ele pode pausar goroutines individuais, inspecionar canais e examinar o estado do garbage collector.

```
// main.go
package main

import "fmt"

func processarDados(dados []int) int {
    resultado := 0
    for i, d := range dados {
        resultado += d / dados[i+1] // Bug: index out of range
    }
    return resultado
}

func main() {
    dados := []int{10, 20, 30, 0}
    fmt.Println(processarDados(dados))
}
```

O OMP detecta o panic e lança uma sessão dlv. O agente pode, então, listar todas as goroutines em execução, inspecionar o frame onde o panic aconteceu e ver o valor de `i` e `dados[i+1]` no momento exato do erro.

As 28 operações DAP em detalhe

O DAP define um conjunto completo de operações que o OMP implementa.

Categoria	Operações	O que fazem
Sessão	initialize, launch, attach, disconnect	Criam e encerram a sessão de debug
Breakpoints	setBreakpoints, setFunctionBreakpoints, setInstructionBreakpoints, setExceptionBreakpoints, clearBreakpoints	Definem onde o programa deve pausar
Controle	continue, next, stepIn, stepOut, stepBack, reverseContinue, pause	Movem a execução pelo código
Estado	threads, stackTrace, scopes, variables, modules	Examinam o estado interno do programa
Avaliação	evaluate	Rodam expressões no contexto do programa
Exceção	setExceptionBreakpoints	Configuram pausa em erros
Dados	dataBreakpoint, instructionBreakpoints	Breakpoints em mudança de dados

Essas 28 operações dão ao agente controle total sobre o programa.

A falha silenciosa em produção

Você está trabalhando em uma API Go que processa pedidos de um e-commerce. Em desenvolvimento, tudo funciona perfeitamente. Mas em produção, o serviço começa a travar aleatoriamente — sem erro, sem log, simplesmente para de responder.

Você suspeita de um deadlock entre goroutines, mas como encontrar o ponto exato onde os canais travam?

O erro comum seria adicionar `fmt.Println` por todo o código, compilar, rodar novamente e torcer para capturar o momento do travamento. É como tentar encontrar um parafuso solto em um navio em movimento, chutando cada equipamento até ouvir um barulho diferente.

A prática correta com o OMP é diferente. O agente se anexa ao processo Go em execução usando `dlv`, sem reiniciar nada. Ele lista todas as goroutines ativas e identifica quais estão bloqueadas em operações de canal.

Em seguida, ele examina os frames de cada goroutine bloqueada, encontrando o ponto exato onde o canal está esperando uma mensagem que nunca chega — talvez porque uma goroutine morreu silenciosamente antes de enviar.

O resultado é preciso: em vez de adivinhar, você tem o estado completo do programa no momento do travamento.

Armadilhas comuns ao usar DAP

Esquecer símbolos de debug. Ao compilar código C/C++, sem `-g` o lldb não consegue mapear endereços de memória para linhas de código. O agente vê endereços hexadecimais em vez de nomes de funções.

Anexar com atraso. Em processos de alta performance, pausar para inspecionar pode causar timeouts. Use breakpoints condicionais em vez de pausar em todas as iterações.

Avaliar expressões com efeitos colaterais. O DAP permite avaliar expressões arbitrárias, mas chamar funções que modificam estado durante debug pode causar comportamento inesperado. Use `context: "watch"` para avaliações sem efeito colateral.

Próximos Passos

Neste capítulo, você viu como o DAP transforma o OMP de um leitor de código em um debugger autônomo. As 28 operações do protocolo dão ao agente controle total sobre a execução do programa — desde inicializar uma sessão até avaliar expressões no contexto de um breakpoint.

Os três debuggers suportados (lldb-dap, dlvsrv e debugpy) cobrem C, C++, Go e Python, permitindo que o Mestre de Estaleiro Digital inspecione qualquer equipamento do estaleiro com precisão cirúrgica.

O mais importante não é apenas a quantidade de operações, mas a integração: o agente não precisa que você abra um IDE separado, configure um plugin ou copie cole comandos. Tudo acontece dentro do fluxo do OMP.

No próximo capítulo, vamos expandir ainda mais o poder do estaleiro: Subagentes. Você aprenderá a dividir trabalho complexo entre workers paralelos, cada um com seu próprio contexto e resultados tipados.

Acesse a documentação completa: <https://omp.sh/docs>

Siga-nos nas redes sociais para dicas, tutoriais e novidades sobre o Oh My Pi.
